

Operator: Flatten

Phase 1.5 nấc 2 → đầu vào cho Phase 2A (viết operator) và Phase 2D (golden model C).

Operator nguy hiểm nhất trong mạng — không phải vì khó, mà vì sai thì không có thông báo lỗi.

1. Vai trò

Duỗi khối 3 chiều `16 × 3 × 3` thành vector 1 chiều `144` phần tử để đưa vào `Linear`.

Không cộng, không nhân, không có tham số. **Nó chỉ đổi cách đọc cùng một vùng nhớ.** Đúng 144 con số đó, đúng thứ tự đó, chỉ là gọi tên chỉ số khác đi.

Trong ONNX nó hiện ra là node `Reshape` chứ không phải `Flatten` — vì `x.flatten(1)` được dịch thành phép đổi shape tổng quát.

2. Công thức

PyTorch xếp tensor theo **NCHW, row-major**. Phần tử ở vị trí `(c, y, x)` nằm ở chỉ số:

$$\text{flat_idx} = c \times (H \times W) + y \times W + x$$

Đọc từ phải sang: `x` chạy nhanh nhất, rồi tới `y`, `c` chạy chậm nhất. Nghĩa là **duyet hết kênh 0 rồi mới sang kênh 1**.

Ví dụ với `16 × 3 × 3` ($H = W = 3$):

Phần tử	Tính	Chỉ số phẳng
<code>(c=0, y=0, x=0)</code>	$0 + 0 + 0$	0
<code>(c=0, y=2, x=2)</code>	$0 + 6 + 2$	8
<code>(c=1, y=0, x=0)</code>	$9 + 0 + 0$	9
<code>(c=5, y=1, x=2)</code>	$45 + 3 + 2$	50
<code>(c=15, y=2, x=2)</code>	$135 + 6 + 2$	143

Keras xếp theo **NHWC**. Cùng phần tử `(c=5, y=1, x=2)` sẽ nằm ở:

$$y \times (W \times C) + x \times C + c = 1 \times 48 + 2 \times 16 + 5 = 85$$

50 so với 85. Cùng một phần tử, hai layout, hai chỉ số hoàn toàn khác nhau — đó là toàn bộ nội dung của bầy ở mục 6.

3. Mã giả

```
flatten(input[C][H][W], output[C*H*W]):
    idx = 0
    for c = 0 .. C-1:
        for y = 0 .. H-1:
            for x = 0 .. W-1:
                output[idx] = input[c][y][x]
                idx = idx + 1
```

Nhưng trong C, nếu `input` vốn đã là mảng liên tục xếp đúng thứ tự đó thì `flatten` là **no-op** — chỉ cần ép kiểu con trỏ:

```
float *flat = (float *)conv3_out; /* 0 lệnh máy, 0 byte bộ nhớ thêm */
```

Đây là lý do nên giữ layout NCHW xuyên suốt golden model: `flatten` trở thành miễn phí thật sự.

4. Áp dụng vào `SmallCNN`

Bước	Shape	Số phần tử
ra khỏi <code>pool</code> thứ 3	$16 \times 3 \times 3$	144
sau <code>flatten</code>	144	144

144 chính là `in_features` của `fc`. Đây là chỗ quyết định kích thước lớp FC — và là chỗ `DigitCNN` cũ thất bại: nó vào FC với 1.568 phần tử ($32 \times 7 \times 7$), nên `fc1` phình lên 200.704 trọng số, chiếm 97,06% toàn mạng.

Ba lần `MaxPool` là thứ ép $28 \rightarrow 14 \rightarrow 7 \rightarrow 3$. Mỗi lần pool cắt số phần tử đi 4 lần, và chính nó quyết định lớp FC to hay nhỏ.

5. Mức FPGA-friendly

✅ **Miễn phí.** Không phải phép toán, chỉ là cách nối dây / cách đánh địa chỉ. Trên FPGA nó thậm chí không tồn tại như một khối riêng — bộ đếm địa chỉ của lớp FC đọc thẳng từ bộ nhớ đầu ra của `conv3`.

6. Chú ý khi port sang C

Bẫy	Chi tiết
Sai layout NCHW/NHWC	Xem mục 2. Nếu lấy trọng số từ model Keras nhưng flatten theo NCHW, vector 144 bị hoán vị. <code>fc</code> vẫn chạy, vẫn ra 10 số, không crash — accuracy rơi về ~10% (đoán bừa). Đây là lỗi âm thầm nguy hiểm nhất của cả golden model
Cách bắt lỗi trên	In vector 144 phần tử từ PyTorch cho 1 ảnh, in vector 144 từ code C cho đúng ảnh đó, so từng phần tử. Khớp hết thì layout đúng. Đừng chỉ so accuracy cuối
Chiều batch	<code>x.flatten(1)</code> giữ chiều 0 (batch), duỗi từ chiều 1 trở đi. Golden model C chạy 1 ảnh nên không có chiều này — đừng bê nguyên chỉ số từ PyTorch sang
Nhầm <code>flatten</code> với <code>transpose</code>	Flatten không đổi thứ tự dữ liệu, chỉ đổi cách đánh chỉ số. Nếu phải đổi thứ tự thật thì đó là phép khác, và phải nhớ đổi cả trọng số <code>fc</code> tương ứng

7. Tự kiểm tra

Với khối `16 × 3 × 3`, làm trên giấy:

- Phần tử `(c=9, y=2, x=0)` nằm ở chỉ số nào sau flatten?
- Ngược lại: chỉ số phẳng **100** ứng với `(c, y, x)` nào? (chia lấy nguyên và lấy dư)
- Nếu port nhầm sang layout NHWC, chỉ số 47 sẽ chứa phần tử `(c, y, x)` nào thay vì `(5, 0, 2)`?
- `DigitCNN` vào FC với 1.568 phần tử, `SmallCNN` với 144. Tỷ lệ 10,9 lần đó đến từ đâu — bao nhiêu phần do bớt kênh, bao nhiêu phần do thêm một lần pool?

Đáp án

Câu 1 — `(c=9, y=2, x=0)` nằm ở chỉ số nào

```
flat_idx = c × (H × W) + y × W + x = 9 × 9 + 2 × 3 + 0 = 81 + 6 + 0 = 87
```

Câu 2 — Chỉ số phẳng 100 ứng với `(c, y, x)` nào

Giải ngược bằng chia lấy nguyên và lấy dư:

```
c = 100 ÷ 9 = 11      dư r = 100 - 99 = 1
y = 1 ÷ 3 = 0
x = 1 mod 3 = 1
```

****** `(c=11, y=0, x=1)` ****** Kiểm ngược: `11 × 9 + 0 × 3 + 1 = 100` ✓.

Câu 3 — Chỉ số 47 đọc nhầm theo NHWC

Trước hết, chỉ số 47 **thật sự** là phần tử nào theo NCHW:

```
c = 47 ÷ 9 = 5      dư 47 - 45 = 2
y = 2 ÷ 3 = 0
```

```
x = 2 mod 3 = 2
→ (c=5, y=0, x=2)
```

Nếu port nhầm và đọc cùng chỉ số 47 theo công thức NHWC $y \times (W \times C) + x \times C + c$ (với $W=3, C=16$):

```
y = 47 ÷ 48 = 0          dư 47
x = 47 ÷ 16 = 2
c = 47 mod 16 = 15
→ hiểu nhầm thành (c=15, y=0, x=2)
```

Cùng một ô nhớ, hai cách hiểu: **kênh 5 thành kênh 15**. Ô nhớ đọc đúng, nhãn gán sai, nên mỗi trọng số của `fc` nhân với sai giá trị. Chương trình **không crash**, chỉ ra số sai — đúng loại bẫy ở mục 6.

Câu 4 — Tỷ lệ 10,9 lần đến từ đâu

```
DigitCNN: 32 kênh × 7×7 = 1.568    (2 lần pool: 28 → 14 → 7)
SmallCNN: 16 kênh × 3×3 = 144      (3 lần pool: 28 → 14 → 7 → 3)
```

Tách riêng từng nguyên nhân bằng cách đổi lần lượt:

Bước	Kết quả	Hệ số
Chỉ bớt kênh 32 → 16, giữ 7×7	$16 \times 7 \times 7 = 784$	2 lần
Từ 784, thêm 1 lần pool (7×7 → 3×3)	$16 \times 3 \times 3 = 144$	5,44 lần
		tích ≈ 10,9

Phần đóng góp lớn hơn là thêm một lần pool (5,44×), không phải bớt kênh (2×) — vì pool tác động lên **cả hai** chiều không gian cùng lúc, còn bớt kênh chỉ tác động một chiều.

Chú ý con số 5,44 chứ không phải 4: $7 \times 7 = 49$ xuống $3 \times 3 = 9$ là chia 5,44, vì `floor` cắt hàng và cột cuối. Đây đúng con số đã gặp ở `relu.md` khi đổi thứ tự `relu ↔ maxpool`.